

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

ESCUELA DE INGENIERÍA

DEPARTAMENTO DE COMPUTACIÓN, ELECTRÓNICA
Y MECATRÓNICA



Lab report #3

Course: Digital design LRT2022-7

Equipo: 2

183339 Juan Pablo Lopez Moreno LMT
185406 Amy Marianee Ramírez Sánchez LMT

April 1st, 20, San Andrés Cholula, Puebla

1 Abstract

This report will present the results obtained from the third laboratory practice of the Digital Design course. The main objective of this practice was to continue to familiarize ourselves with the functioning of logic gates and boolean functions. Showcasing both a program in VHDL and simulation function based on logic gates.

2 Introduction

The analysis and design of digital circuits is based on the representation and manipulation of Boolean functions through logic gates. These gates allow the implementation of fundamental operations in digital electronics, and when combined, enable the construction of complex systems. Therefore, it is essential to understand both the algebraic expressions and their translation into schematics, as well as the reverse process of generating schematics from Boolean functions. The main objective of this laboratory practice was to establish the bidirectional relationship between Boolean functions and logic circuits: deriving expressions from schematics and, conversely, constructing schematics from given functions. Additionally, modern digital design tools, such as Multisim simulation and VHDL programming using the EDA Playground platform, were incorporated, allowing for both experimental and virtual validation of circuit functionality.

- NAND, output is false only if all inputs are true.
- NOR, output is true only if any input is false.
- XOR, outputs true if inputs are different.
- XNOR, outputs true if inputs are the same.

[1] It's critical to comprehend not only how these compounded gates operate but also the various ways in which they can be connected within a circuit. Additionally, we can investigate the plethora of possibilities in the field of digital logic by experimenting with various compounded gate configurations.

3 Objectives

The objectives of this lab are:

- Obtain Boolean functions from schematics.
- Obtain schematics from Boolean functions.
- Program and simulate Boolean functions using VHDL.

For the purpose of this lab, it is important to know about the components used, in the first place, "logic gates are defined as simple digital circuits that take one or more binary inputs and produce a binary output"[2]. On the other hand, boolean funtions consists of a number of Boolean variables joined by the Boolean connectives, like AND and OR. [3] Similary to the

previous lab, the software utilized in the experiment is VHDL (Hardware Description Language), which is a language that describes the behavior of electronic circuits, most commonly digital circuits. [4] For this experiment, it is fundamental to explain the procedure to convert a schematic into a boolean function, in this case, one must know how to write a truth table, and construct the canonic form of the function.

To understand this, we must define the elements of a electronic schematic. For most circuits, a photograph or realistic drawing showing actual components and their interconnections would be far too complex to be of value in replicating the circuit. A schematic symbol is a simplified representation of a real-world component. A schematic diagram shows such representations of real-world components and a simplified “map” of how they are connected together.[5]

Once a schematic is created or obtained, the truth table must be generated, since the truth table is defined as a canonical representation of a Boolean function that exhaustively enumerates every truth assignment and its corresponding function value. [6]

And it is through this truth table that a canonic function can be obtained, which refers to a Boolean function that is expressed either as a sum of minterms or as a product of maxterms. It represents the standard form of a Boolean function that can be manipulated to simplify its expression. [7]

4 Methodology

The methodology applied in this laboratory practice was structured into three main phases: physical implementation in the laboratory, simulation in Multisim, and programming in EDA Playground.

4.1 Physical Laboratory Phase

During the laboratory work, the following activities were carried out:

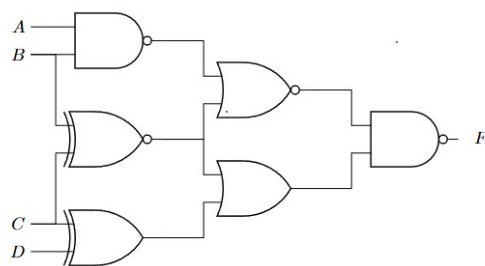


Figure 1: Schematic

Figure 1: Schematic 1, from which the truth table and function are based

- The Boolean function and the truth table corresponding to the schematic shown in Figure 1 were obtained through logical analysis of its structure.
- The schematic and truth table of the Boolean function provided in the practice were determined:

$$F(A, B, C, D) = (B' \cdot D) \cdot (A \oplus B) + (C \cdot A)$$

- The consistency between the derived truth tables and the constructed schematics was verified, ensuring the correct representation of the logical expressions.

4.2 Multisim Simulation Phase

In the Multisim simulation environment, the following procedures were performed:

- Both circuits were virtually constructed using the logic components available in the software.
- Simulations were executed to validate the expected behavior of each circuit.
- The obtained results were compared with the analytically derived truth tables to confirm the correctness of the designs.

4.3 EDA Playground Programming Phase

Finally, in the EDA Playground platform, the following actions were conducted:

- Both Boolean functions were programmed using the VHDL language, following proper coding conventions.
- Testbenches were created and applied to verify the correct functionality of the circuits under all possible input combinations.
- The simulation results were validated against those obtained in the previous phases.

5 Results

5.1 Physical lab experimentation

In this lab, the physical circuits were not built, but based on a schematic, the team developed a truth table and a function:+

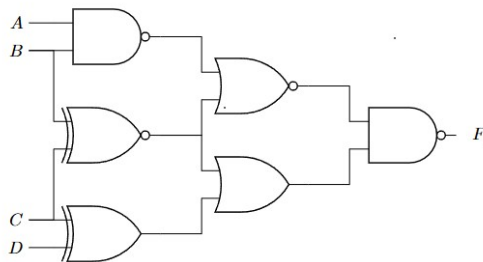


Figure 1: Schematic

Figure 2: Schematic 1, from which the truth table and function are based

After analysing this circuit, the following function was formulated:

$$Y = \overline{((\overline{A \cdot B}) + \overline{(B \oplus C)}) \cdot ((B \oplus C) + (C \oplus D))}$$

And, based on both the function and the schematic, the following truth table was generated:

Input				Output
A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Table 1: Truth Table for the first schematic provided

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

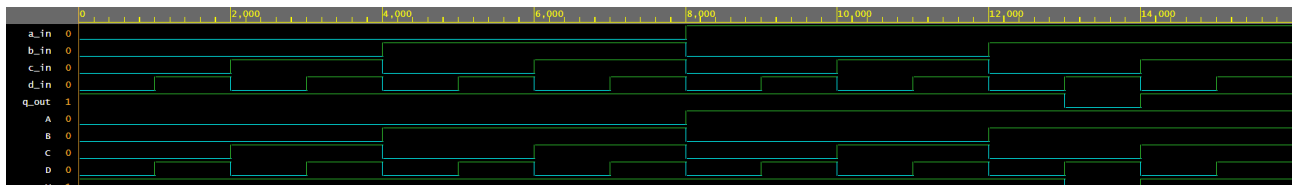


Figure 3: Wave made from the function of the Schematic 1.

For the second experiment, the team was given a function, with the mission of obtaining the schematic and truth table:

$$F(A, B, C, D) = (\bar{B} \cdot D) \cdot (A \oplus B) + (\overline{C \cdot A})$$

And after analysing the function, the following schematic was generated

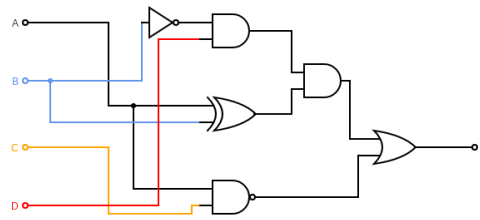


Figure 4: Schematic 2, based on the function given.

And, based on both the function and the schematic, the following truth table was generated:

Input				Output
A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

Table 2: Truth Table for the first schematic provided

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

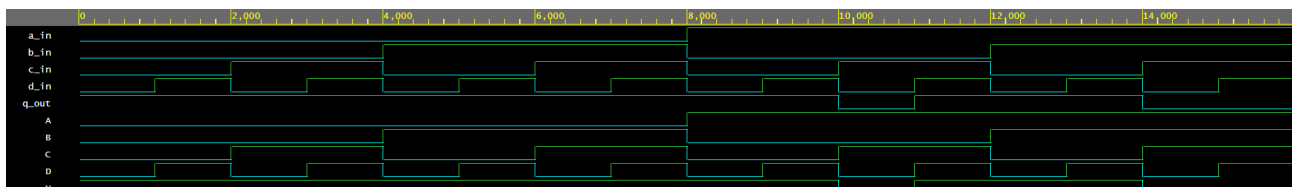


Figure 5: Wave made from the function of the Schematic 2.

5.2 Simulation

Based on the schematics of the previous experiments, the following simulation was built on MultiSim:

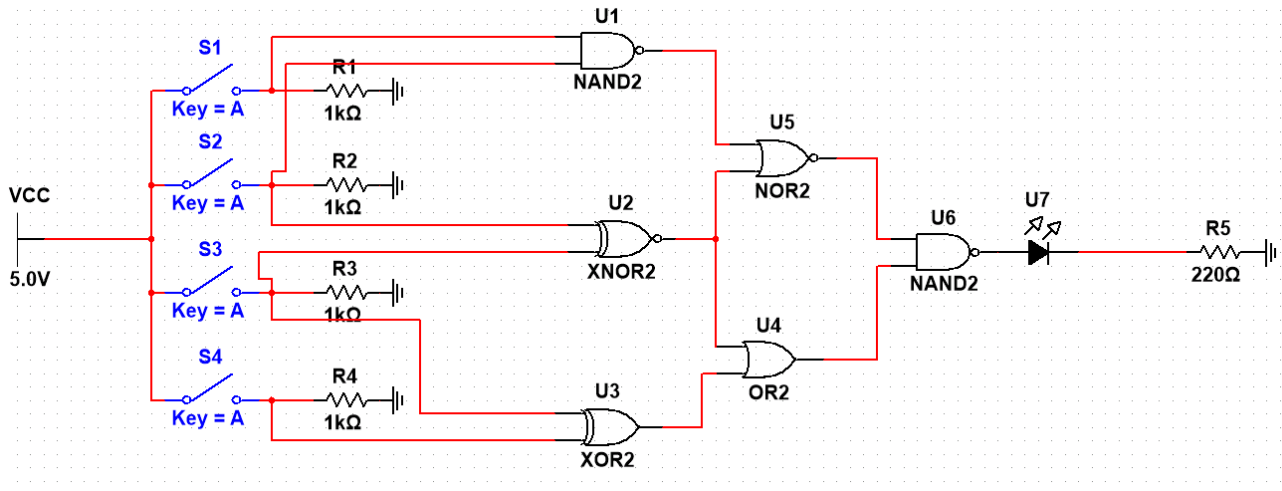


Figure 6: MultiSim schematics made for the first schematic.

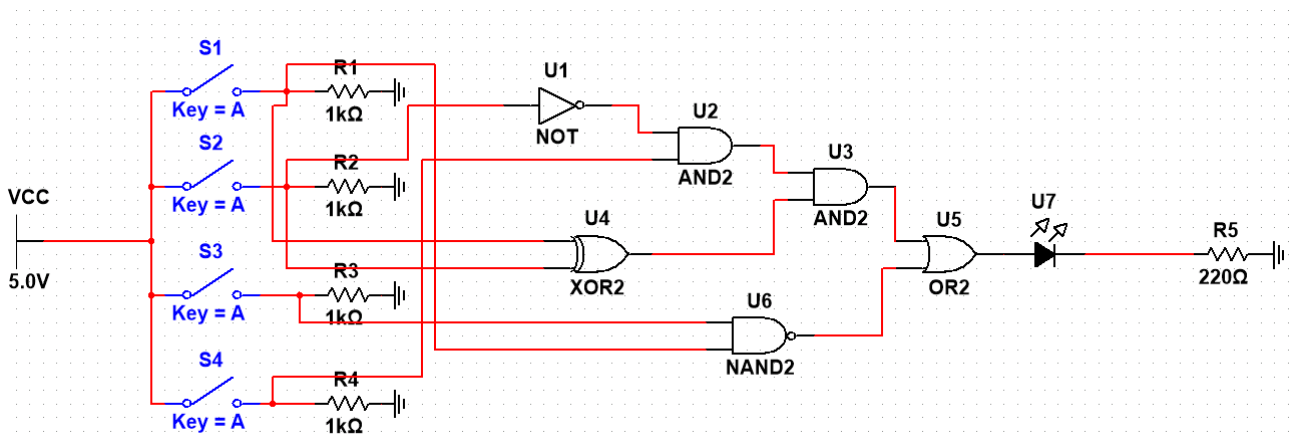


Figure 7: MultiSim schematics made for the second schematic.

5.3 Codes

Finally, in order to generate the previous Waves and to prove the truth tables, the team created the following codes:

Código 1: Code 1: For exercise 1

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration
```

```
entity ent_lab_3 is

    port(A : in std_logic;      — OR gate input
          B : in std_logic;
          C : in std_logic;
          D : in std_logic;
          Y : out std_logic);   — OR gate output

end ent_lab_3;

— Architecture definition

architecture arq_lab_3 of ent_lab_3 is

    begin

        Y <= (((A nand B)nor(B xnor C))nand((B xnor C)or(C xor D)));

    end arq_lab_3;
```

Código 2: Code 2: For exercise 2

```
library IEEE;
use IEEE.std_logic_1164.all;

— Entity declaration

entity ent_lab_3_2 is

    port(A : in std_logic;
          B : in std_logic;
          C : in std_logic;
          D : in std_logic;
          Y : out std_logic);

end ent_lab_3_2;

— Architecture definition

architecture arq_lab_3_2 of ent_lab_3_2 is

    begin

        Y <= (((not(B)and D)and(A xor b))or(C nand A));

    end arq_lab_3_2;
```

And to test this codes, the team created the following:

Código 3: Code 1 and 2 Testbench

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component ent_lab_3_2 is
port(
  A: in std_logic;
  B : in std_logic;
  C : in std_logic;
  D : in std_logic;
  Y: out std_logic);
end component;

signal a_in, b_in, c_in, d_in, q_out: std_logic;

begin

-- Connect UUT
UUT: ent_lab_3_2 port map(a_in, b_in, c_in, d_in, q_out);

process
begin

  a_in <= '0';
  b_in <= '0';
  c_in <= '0';
  d_in <= '0';
  wait for 1 ns;

  a_in <= '0';
  b_in <= '0';
  c_in <= '0';
  d_in <= '1';
  wait for 1 ns;

  a_in <= '0';
  b_in <= '0';
```

```
c_in <= '1';  
d_in <= '0';  
wait for 1 ns;
```

```
a_in <= '0';  
b_in <= '0';  
c_in <= '1';  
d_in <= '1';  
wait for 1 ns;
```

```
a_in <= '0';  
b_in <= '1';  
c_in <= '0';  
d_in <= '0';  
wait for 1 ns;
```

```
a_in <= '0';  
b_in <= '1';  
c_in <= '0';  
d_in <= '1';  
wait for 1 ns;
```

```
a_in <= '0';  
b_in <= '1';  
c_in <= '1';  
d_in <= '0';  
wait for 1 ns;
```

```
a_in <= '0';  
b_in <= '1';  
c_in <= '1';  
d_in <= '1';  
wait for 1 ns;
```

```
a_in <= '1';  
b_in <= '0';  
c_in <= '0';  
d_in <= '0';  
wait for 1 ns;
```

```
a_in <= '1';  
b_in <= '0';  
c_in <= '0';  
d_in <= '1';  
wait for 1 ns;
```

```
a_in <= '1';
b_in <= '0';
c_in <= '1';
d_in <= '0';
wait for 1 ns;

a_in <= '1';
b_in <= '0';
c_in <= '1';
d_in <= '1';
wait for 1 ns;

a_in <= '1';
b_in <= '1';
c_in <= '0';
d_in <= '0';
wait for 1 ns;

a_in <= '1';
b_in <= '1';
c_in <= '0';
d_in <= '1';
wait for 1 ns;

a_in <= '1';
b_in <= '1';
c_in <= '1';
d_in <= '0';
wait for 1 ns;

a_in <= '1';
b_in <= '1';
c_in <= '1';
d_in <= '1';
wait for 1 ns;

end process;
end tb;
```

6 Analysis of Results

6.1 Detailed Analysis of the Figure and Determination of the Logic Function

6.1.1 Gate Identification and Description (According to Figure 1)

By carefully examining the provided figure, each logic block was identified and how they interact to form the final output was established. The gates are described as follows:

- **Top left gate: NAND with inputs A and B.**

$$N = (A \cdot B)'$$

This gate produces a high output for all input combinations, except when A and B are both 1, where the output is 0. It plays a key role in controlling the upper right section of the circuit.

- **Middle left gate: XNOR with inputs B and C, with a negation bubble at the output.**

$$M = B \text{ XNOR } C = (B \oplus C)'$$

The output is high when B and C are equal and low when they differ. The bubble indicates that the XOR operation is reversed, producing XNOR behavior.

- **Lower left gate: XOR with inputs C and D, with no negation on the output.**

$$X = C \oplus D$$

The output is high only when C and D are different, introducing variability depending on the relationship between the two inputs.

- **Upper center-right gate: NOR with N and M inputs.**

$$T = (N + M)'$$

The output T will be high only when both inputs N and M are low, acting as a detector for conditions where the previous signals are zero and affecting the final output.

- **Bottom center-right gate: OR with inputs M and X.**

$$B_2 = M + X$$

Output B_2 will be high if at least one of the inputs is 1, ensuring that partial conditions detected by M or X activate the signal.

- **Final gate (right): NAND with inputs T and B_2 , producing output Y.**

$$F_1 = (T \cdot B_2)'$$

The final output is high in most combinations and only goes low when T and B_2 are simultaneously high, representing the critical condition of the circuit.

Therefore, the final output is:

$$Y = (((A \cdot B)' + (B \oplus C)')' \cdot ((B \oplus C)' + (C \oplus D)))'$$

6.1.2 Compound Boolean Expression and Step-by-Step Analysis of the Output

Replacing the above definitions in the final output:

$$Y = (T \cdot B_2)'$$

Conditions for $Y = 0$ For the Y output to be 0, both inputs of the final NAND must be high:

$$T = 1 \text{ and } B_2 = 1$$

- **Analyzing T :**

$$T = (N + M)'$$

For $T = 1$, the inner sum must be 0:

$$N + M = 0 \Rightarrow N = 0 \text{ and } M = 0$$

- **Analyzing B_2 :**

$$B_2 = M + X$$

For $B_2 = 1$, at least one input must be high:

$$M + X = 1 \Rightarrow X = 1 \text{ (since } M = 0)$$

Translation to Input Combinations

- $N = (A \cdot B)' = 0 \rightarrow A \cdot B = 1 \rightarrow A = 1 \text{ and } B = 1$
- $M = (B \oplus C)' = 0 \rightarrow B \oplus C = 1 \rightarrow B \text{ and } C \text{ are different, with } B = 1 \rightarrow C = 0$
- $X = C \oplus D = 1 \rightarrow C \neq D$, with $C = 0 \rightarrow D = 1$

The only combination that satisfies all the conditions is:

$$(A, B, C, D) = (1, 1, 0, 1)$$

Thus, $Y = A \cdot B \cdot C' \cdot D$, and its minterms representation is decimal row 13 (binary 1101). In maxterms product:

$$Y = \prod M(13)$$

6.1.3 Verification Using Truth Tables and Simulation

- Table 1 confirms that the output is 1 for 15 of the 16 possible input combinations, perfectly matching the algebraic analysis.
- The only row with $Y = 0$ corresponds exactly to the combination $(1, 1, 0, 1)$, as deduced step by step.
- Simulation in Multisim and EDA Playground, represented in waveforms, shows that the output remains high for most input vectors and presents a single low pulse for the indicated combination, verifying the consistency between the truth table, the Boolean expression, and the practical simulation.

6.2 Detailed Analysis of the Function and Determination of the Logic

6.2.1 Identification and Description of the Gates (According to the Provided Function)

The proposed function is:

$$F(A, B, C, D) = (B' \cdot D) \cdot (A \oplus B) + (A \cdot C)$$

Analyzing each logic block:

- **NOT gate at B:** B' Inverts the value of input B, enabling conditions dependent on B being 0.
- **AND gate between B' and D:** $E = B' \cdot D$ The output E will be high when $B = 0$ and $D = 1$.
- **XOR gate between A and B:** $X = A \oplus B$ This output is high only when A and B are different.
- **AND gate between E and X:** $T = (B' \cdot D) \cdot (A \oplus B)$ This block detects the specific condition: $B = 0$, $D = 1$, and $A \neq B$.
- **AND gate between A and C:** $U = A \cdot C$ Activates the output when $A = 1$ and $C = 1$, regardless of B or D.
- **Final OR gate between T and U:** $Y = T + U$ The output will be high if either of the two conditions is met: the combination of the first branch (T) or the direct condition $A \cdot C$.

Thus, the output is described as:

$$Y = (B' \cdot D) \cdot (A \oplus B) + (A \cdot C)$$

Once the analysis of the function was completed, the diagram that reflects its logical behavior was developed, which facilitated the interpretation and verification of the obtained result.

6.2.2 Compound Boolean Expression and Step-by-Step Analysis of the Output

Conditions for Output = 1 The output is 1 when at least one branch of the OR is 1:

- **Branch 1 (T):**

$$(B' \cdot D) \cdot (A \oplus B) = 1$$

This requires simultaneously:

- $B = 0$ and $D = 1$
- $A \neq B$ (i.e., $A = 1$)

Therefore, Branch 1 is activated with: $(A, B, C, D) = (1, 0, *, 1)$ (C is irrelevant in this branch).

- **Branch 2 (U):**

$$A \cdot C = 1$$

This requires $A = 1$ and $C = 1$, regardless of B and D.

Translation to Input Combinations

- From Branch 1, we obtain minterms: $(1, 0, 0, 1)$ and $(1, 0, 1, 1) \rightarrow$ decimals 9 and 11.
- From Branch 2, we obtain minterms:
 - $(1, *, 1, 0) \rightarrow (1, 0, 1, 0)$ and $(1, 1, 1, 0) \rightarrow$ decimals 10 and 14
 - $(1, *, 1, 1) \rightarrow (1, 0, 1, 1)$ and $(1, 1, 1, 1) \rightarrow$ decimals 11 and 15

Joining results:

$$Y = \sum m(9, 10, 11, 14, 15)$$

6.2.3 Verification via Truth Table and Simulation

- Table 2 confirms that the output is 1 for 5 of the 16 possible input combinations, perfectly matching the algebraic analysis.
- Simulation in Multisim and EDA Playground, represented in waveforms, shows that the output remains high for most input vectors and presents a single low pulse for the indicated combination, verifying the consistency between the truth table, the Boolean expression, and the practical simulation.

This detailed analysis confirms that the circuit design correctly implements the intended logic function. The methodology (gate identification, partial output analysis, and verification of the conditions for $Y = 0$) ensures accurate results and provides a complete understanding of the role of each block within the circuit.

7 Conclusions

The development of this practice allowed the fulfillment of the laboratory objectives, as Boolean functions were obtained from schematics, and schematics were interpreted from Boolean functions. Additionally, the Boolean functions were programmed and simulated using VHDL, confirming their correct logical behavior. The detailed identification of gates, analysis of partial outputs, and the creation of truth tables and canonical forms allowed algebraic validation of the results. Simulations performed in Multisim and EDA Playground corroborated the consistency between theory and practice, demonstrating the usefulness of digital simulation tools for predicting and verifying circuit operation. In this way, the practice not only enabled experimental validation of theoretical principles but also reinforced the understanding of programming and simulation of Boolean functions in digital environments.

References

- [1] Unacademy, “Logic gates,” <https://unacademy.com/content/jee/study-material/physics/logic-gates>, accessed: [Current Date].
- [2] S. L. Harris and D. Harris, “1 - from zero to one,” in *Digital Design and Computer Architecture*, S. L. Harris and D. Harris, Eds. Morgan Kaufmann, 2022, pp. 1–50. [En línea]. Disponible: <https://www.sciencedirect.com/science/article/pii/B9780128200643000015>
- [3] B. HOLDSWORTH and R. WOODS, “2 - boolean algebra,” in *Digital Logic Design (Fourth Edition)*, fourth edition ed., B. HOLDSWORTH and R. WOODS, Eds. Oxford: Newnes, 2002, pp. 28–42. [En línea]. Disponible: <https://www.sciencedirect.com/science/article/pii/B9780750645829500032>
- [4] Intel Corporation, “Vhdl definition,” https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm, 2017, accessed: 2024-10-27.
- [5] J. Varteresian, “Chapter 1 - schematic capture,” in *Fabricating Printed Circuit Boards*, J. Varteresian, Ed. Woburn: Newnes, 2002, pp. 1–10. [En línea]. Disponible: <https://www.sciencedirect.com/science/article/pii/B9781878707505500022>
- [6] J.-H. R. Jiang and S. Devadas, “Chapter 6 - logic synthesis in a nutshell,” in *Electronic Design Automation*, L.-T. Wang, Y.-W. Chang, and K.-T. T. Cheng, Eds. Boston: Morgan Kaufmann, 2009, pp. 299–404. [En línea]. Disponible: <https://www.sciencedirect.com/science/article/pii/B9780123743640500138>
- [7] B. HOLDSWORTH and R. WOODS, “3 - karnaugh maps and function simplification,” in *Digital Logic Design (Fourth Edition)*, fourth edition ed., B. HOLDSWORTH and R. WOODS, Eds. Oxford: Newnes, 2002, pp. 43–80. [En línea]. Disponible: <https://www.sciencedirect.com/science/article/pii/B9780750645829500044>